

Cahier des Charges : Modules de Monitoring et d'Analyse NIDS

Ce document se concentre sur l'architecture des flux de données et l'interface utilisateur, en s'appuyant sur les spécifications techniques initiales.

A. Contexte et Objectifs Spécifiques

L'objectif de ces sous-modules est de créer une chaîne de traitement réactive capable de remonter l'information depuis l'interface réseau jusqu'à l'écran de l'administrateur avec une latence quasi nulle. Il s'agit de relier le moteur NIDS (Zeek/Suricata) à l'interface Web via une API robuste et un pipeline de streaming.

B. Besoins Fonctionnels par Module

1. Module "TrafficCapture" (Capture du Trafic)

- Le système doit écouter et capturer le trafic réseau en s'interfaçant directement avec Zeek ou Suricata.
- Le module doit supporter l'ingestion de flux en temps réel ainsi que la lecture de fichiers .pcap pour des analyses a posteriori.
- Les métadonnées extraites des paquets (IP source/destination, ports, protocoles, taille) doivent être envoyées vers le pipeline de streaming (Redis Streams) pour un traitement asynchrone.

2. Module "AnalyzeTraffic" (Analyse et Inférence IA)

- Le module doit consommer les données brutes depuis Redis Streams et les formater pour l'inférence.
- Il doit transmettre ces caractéristiques (features) au modèle d'IA (TensorFlow/Scikit-Learn), qui agit comme un microservice Python indépendant.
- Une fois l'analyse terminée, le module doit récupérer la prédiction (Trafic Normal, DoS, Port Scanning) et attribuer un score de confiance.

3. Module "API" (Backend Node.js/Express)

- L'API doit centraliser la communication entre la base de données (MongoDB), le pipeline d'analyse et le frontend.
- Elle doit exposer des routes REST sécurisées pour la consultation de l'historique des alertes et des statistiques globales.

- L'accès aux endpoints doit être strictement protégé par une authentification JWT et un contrôle d'accès basé sur les rôles (RBAC).
- L'API doit maintenir un serveur WebSockets (Socket.io) pour pousser instantanément (push) les nouvelles détections vers les clients connectés.

4. Module "Dashboard" (Interface Web React.js)

- Le Dashboard doit être développé en React.js avec TypeScript et TailwindCSS pour une interface moderne et réactive.
- Il doit se connecter au flux WebSockets de l'API pour mettre à jour les statistiques et les graphiques sans rafraîchir la page.
- Le tableau de bord interactif doit obligatoirement afficher : l'adresse IP source, le type d'attaque identifiée et l'heure de l'incident (Timestamp).
- L'interface doit générer des alertes visuelles instantanées (bannières, codes couleurs rouges) lorsqu'une menace critique est détectée.

C. Exigences de Performance et Flux de Données

- Latence UI : Le délai entre la détection d'une anomalie par le microservice Python et l'affichage de l'alerte sur le Dashboard React.js ne doit pas dépasser 500 millisecondes.
- Résilience : En cas de déconnexion du flux WebSocket, le Dashboard doit tenter une reconnexion automatique tout en récupérant les alertes manquées via une requête REST classique à l'API.